topic_6_microservices_architecture

Theory, Edge Cases & Code Examples

- ✓ What is Microservices Architecture?
- ✓ Simple Explanation: City vs Village Analogy
- ✓ Monolithic vs Microservices
- ✓ Monolithic Application
- ✓ Microservices Application

→ Swipe to learn →

topic_6_microservices_architecture

Theory

📌 What is Microservices Architecture?

Microservices Architecture is a design pattern where you build an application as a **collection of small, independent services** that each run in their own process and communicate over a network (usually HTTP/REST or message queues).

Key Idea: Instead of one big application (monolith), break it into many small services, each doing ONE thing well.

---

topic_6_microservices_architecture

Theory

📌 Simple Explanation: City vs Village Analogy

Monolithic (Village)	Microservices (City)
One general store sells everything	**Specialized shops**: bakery, butcher, pharmacy
Everyone goes to same place	Each shop focuses on ONE thing
If store closes, village has nothing	If bakery closes, butcher still works
Hard to expand (need bigger building)	Easy to expand (add more shops)
One owner manages everything	Each shop has own owner

Microservices: Like a city with specialized shops instead of one store handling everything.

---

topic_6_microservices_architecture

Theory

📌 Monolithic vs Microservices

topic_6_microservices_architecture

Theory

📌 Monolithic Application

```
┌──────────────────────────────────────────┐
│          One Big Application             │
│                                          │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐│
│   │  Users   │   │Products  │   │ Orders   │
│   │ Module   │   │ Module   │   │ Module   │
│   └──────────┘   └──────────┘   └──────────┘│
│                                          │
│          All in one codebase             │
│          One database                    │
│          Deployed together               │
└──────────────────────────────────────────┘
```

Problems:
- ❌ **Can't scale independently** (must scale entire app)
- ❌ **One bug can crash everything**
- ❌ **Long deployment times** (entire app must be redeployed)
- ❌ **Technology lock-in** (entire app uses same language/framework)
- ❌ **Large codebase** (hard to understand)

---

topic_6_microservices_architecture

Theory

📌 Microservices Application

```
┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│   User       │     │  Product     │     │   Order      │
│  Service     │     │  Service     │     │  Service     │
│              │     │              │     │              │
│  Port 8001   │     │  Port 8002   │     │  Port 8003   │
│              │     │              │     │              │
│  User DB     │     │ Product DB   │     │  Order DB    │
└──────────────┘     └──────────────┘     └──────────────┘

        ↑                    ↑                    ↑
        └────────────────────┴────────────────────┘
                             |
                 HTTP REST / Message Queue
```

Benefits:
- ✅ **Scale independently** (scale only Order Service if needed)
- ✅ **Fault isolation** (if User Service crashes, Product Service still works)
- ✅ **Fast deployment** (deploy only changed service)
- ✅ **Technology diversity** (User Service in C++, Order Service in Python)
- ✅ **Small, manageable codebases**

---

topic_6_microservices_architecture

Theory

📌 Microservices Characteristics

topic_6_microservices_architecture

Theory

📌 1. Single Responsibility

Each service does **ONE thing well**.

```
✅ GOOD:
- UserService: Handles user authentication and profiles
- OrderService: Handles order creation and tracking
- PaymentService: Handles payment processing

❌ BAD:
- BackendService: Handles users, orders, payments, products, shipping... (monolith!)
```

---

topic_6_microservices_architecture

Theory

📌 2. Independent Deployment

Each service can be **deployed independently** without affecting others.

```
Scenario: Fix bug in User Service
Monolith: Deploy ENTIRE application (risk to all features)
Microservices: Deploy ONLY User Service (no risk to Orders, Products, etc.)
```

---

topic_6_microservices_architecture

Theory

📌 3. Decentralized Data

Each service has its **own database** (no shared database!).

```
┌──────────────┐          ┌──────────────┐
│   User       │          │   Order      │
│  Service     │          │  Service     │
│              │          │              │
│  ┌────────┐  │          │  ┌────────┐  │
│  │User DB│  │          │  │OrderDB│  │
│  └────────┘  │          │  └────────┘  │
└──────────────┘          └──────────────┘

❌ NOT like this:
┌──────────────┐          ┌──────────────┐
│   User       │          │   Order      │
│  Service     │          │  Service     │
└──────────────┘          └──────────────┘
       ↓                         ↓
       └──────────┬──────────────┘
                  │
            ┌─────────┐
            │Shared   │
            │  DB     │  ← WRONG! Creates coupling
            └─────────┘
```

Why? Each service can change its database schema without affecting others.

---

topic_6_microservices_architecture

Theory

📌 4. Communicate via APIs

Services talk to each other over **network** (HTTP REST, gRPC, message queues).

```
// User Service wants to get order history
// ❌ WRONG: Direct database access
OrderDatabase::query("SELECT * FROM orders WHERE user_id = " + userId);

// ✅ CORRECT: Call Order Service API
auto response = httpClient.get("http://order-service:8003/api/orders?userId=" + userId);
```

---

topic_6_microservices_architecture

Theory

📌 Communication Patterns

topic_6_microservices_architecture

Theory

📌 1. Synchronous (HTTP/REST)

Request-Response: Client waits for response.

```
User Service                    Order Service
     |                                |
     |  GET /orders?userId=123        |
     |──────────────────────────────>|
     |                                |
     |        [200 OK] {orders: [...]} |
     |<──────────────────────────────|
     |                                |
```

Pros: Simple, immediate response
Cons: Slow if service down, tight coupling

Example:

```
std::string getUserOrders(int userId) {
    // HTTP call to Order Service
    HttpClient client;
    auto response = client.get("http://order-service/api/orders?userId=" + std::to_string(userId));
    return response.body();
}
```

---

topic_6_microservices_architecture

Theory

📌 2. Asynchronous (Message Queue)

Publish-Subscribe: Fire and forget, don't wait.

```
User Service                    Message Queue              Order Service
     |                               |                          |
     |  Publish: UserRegistered      |                          |
     |──────────────────────────────>|                          |
     |                               |                          |
     |  (doesn't wait)               |    Subscribe & Process   |
     |                               |──────────────────────────>|
     |                               |                          |
```

Pros: Fast (no waiting), loose coupling
Cons: Eventual consistency, harder to debug

Example:

```
void registerUser(const User& user) {
    // Save user locally
    userRepository.save(user);

    // Publish event (don't wait for response)
    messageQueue.publish("UserRegistered", {
        {"userId", user.id},
        {"email", user.email}
    });

    // Other services (Email Service, Analytics Service) will receive event
}
```

---

topic_6_microservices_architecture

Theory

📌 API Gateway Pattern (1/2)

Problem: Client needs to call multiple services → many network calls, complex client code.

```
Mobile App
    │
    ├─> User Service
    ├─> Order Service
    ├─> Product Service
    └─> Payment Service

4 network calls! Slow! Complex!
```

Solution: API Gateway acts as single entry point.

```
Mobile App
    │
    └─> API Gateway
            │
            ├─> User Service
            ├─> Order Service
            ├─> Product Service
            └─> Payment Service

1 network call! Fast! Simple!
```

API Gateway Responsibilities:
- **Routing**: Forward requests to correct service
- **Aggregation**: Combine responses from multiple services
- **Authentication**: Check user credentials once
- **Rate limiting**: Prevent abuse

Example:

```
class APIGateway {
public:
    Response handleRequest(Request request) {
        // Authenticate
        if (!authService.verify(request.token)) {
            return Response{401, "Unauthorized"};
// ...continues on next slide
```

📌 API Gateway Pattern (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        }

        // Route to service
        if (request.path == "/api/users") {
            return forwardTo("user-service", request);
        } else if (request.path == "/api/orders") {
            return forwardTo("order-service", request);
        }

        return Response{404, "Not Found"};
    }

private:
    Response forwardTo(const std::string& serviceName, Request request) {
        std::string serviceURL = serviceDiscovery.getURL(serviceName);
        return httpClient.request(serviceURL, request);
    }
};
```

topic_6_microservices_architecture

Theory

📌 Service Discovery (1/2)

Problem: Services don't know where other services are (IP addresses change).

Solution: Service Registry keeps track of all services.

```
┌──────────────────────────────────────┐
│        Service Registry              │
│  (Consul, Eureka, etcd)              │
│                                      │
│  user-service:    192.168.1.10:8001  │
│  order-service:   192.168.1.11:8002  │
│  product-service: 192.168.1.12:8003  │
└──────────────────────────────────────┘
         ↑                    ↑
         | register           | lookup
         |                    |
┌────────┴─────────┐    ┌────────┴─────────┐
│  User Service    │    │ Order Service    │
│                  │    │                  │
│ On startup:      │    │ Want to call     │
│ "I'm at :8001!"  │    │ User Service     │
└──────────────────┘    └──────────────────┘
```

Flow:

1. Service starts → **registers** with registry ("I'm User Service at 192.168.1.10:8001")
2. Other service wants to call it → **looks up** registry ("Where is User Service?")
3. Registry responds: "User Service is at 192.168.1.10:8001"

Example:

```
class ServiceRegistry {
    std::map<std::string, std::string> services;  // name -> URL

public:
    void register(const std::string& serviceName, const std::string& url) {
        services[serviceName] = url;
        std::cout << serviceName << " registered at " << url << "\n";
    }

    std::string lookup(const std::string& serviceName) {
// ...continues on next slide
```

topic_6_microservices_architecture

Theory

📌 Service Discovery (2/2)

[Code continues across slides]

```
// ...continued from previous slide
        if (services.find(serviceName) != services.end()) {
            return services[serviceName];
        }
        throw std::runtime_error("Service not found: " + serviceName);
    }
};

// User Service on startup
void main() {
    ServiceRegistry registry;
    registry.register("user-service", "http://192.168.1.10:8001");

    // Start server...
}

// Order Service wants to call User Service
std::string callUserService(int userId) {
    std::string url = registry.lookup("user-service");
    return httpClient.get(url + "/api/users/" + std::to_string(userId));
}
```

topic_6_microservices_architecture

Theory

📌 When to Use Microservices

Use When	Don't Use When
✅ Large teams (50+ developers)	❌ Small team (< 5 people)
✅ Need independent scaling	❌ Simple CRUD application
✅ Different tech stacks needed	❌ Startup (rapid changes, unclear boundaries)
✅ High availability critical	❌ Limited DevOps resources
✅ Clear domain boundaries	❌ Tightly coupled features

Golden Rule: **Start with a monolith, split into microservices when you NEED to.**

---

topic_6_microservices_architecture

Theory

📌 Benefits vs Challenges

Benefits	Challenges
✅ Independent scaling	❌ Distributed system complexity
✅ Technology diversity	❌ Network latency and failures
✅ Fault isolation	❌ Debugging is harder
✅ Faster deployment	❌ Data consistency issues
✅ Team autonomy	❌ Need DevOps expertise

topic_6_microservices_architecture

Theory

📌 Real-World Examples

Netflix:
- 700+ microservices
- Each service owns its data
- Polyglot architecture (Java, Node.js, Python, Go)
- Handle 200 million+ users

Amazon:
- Thousands of microservices
- "Two-pizza teams" (small, autonomous teams)
- Each service < 1000 lines of code

Uber:
- 2,200+ microservices
- Services for: routing, pricing, driver matching, payments, etc.
- Each service independently deployable

---

topic_6_microservices_architecture

Edge Cases

📌 1. Distributed Transactions (SAGA Pattern) (1/3)

Problem: Transaction spans multiple services. What if one fails?

Scenario: User places order → need to:
1. Reserve inventory (Product Service)
2. Charge credit card (Payment Service)
3. Create order (Order Service)

Naive Code:

```
void placeOrder(int userId, int productId, double amount) {
    // Call Service 1
    productService.reserveInventory(productId);

    // Call Service 2
    paymentService.chargeCard(userId, amount);  // ❌ What if this fails?

    // Call Service 3
    orderService.createOrder(userId, productId);

    // If ANY service fails, how to rollback?
}
```

Why It Fails:
- Inventory reserved, but payment failed → inventory stuck as reserved!
- No distributed transaction coordinator
- Can't use database ACID transactions (each service has own DB)

Solution: SAGA Pattern (Choreography)

Each service publishes events, others react:

```
// Order Service
void placeOrder(int userId, int productId, double amount) {
    Order order = {userId, productId, "PENDING"};
    orderRepository.save(order);

    messageQueue.publish("OrderCreated", {
        {"orderId", order.id},
// ...continues on next slide
```

📌 1. Distributed Transactions (SAGA Pattern) (2/3)

[Code continues across slides]

```
// ...continued from previous slide
        {"productId", productId},
        {"amount", amount}
    });
}

// Product Service (listens to OrderCreated)
void onOrderCreated(Event event) {
    try {
        reserveInventory(event.productId);
        messageQueue.publish("InventoryReserved", {
            {"orderId", event.orderId}
        });
    } catch (...) {
        messageQueue.publish("InventoryReservationFailed", {
            {"orderId", event.orderId}
        });
    }
}

// Payment Service (listens to InventoryReserved)
void onInventoryReserved(Event event) {
    try {
        chargeCard(event.userId, event.amount);
        messageQueue.publish("PaymentSucceeded", {
            {"orderId", event.orderId}
        });
    } catch (...) {
        messageQueue.publish("PaymentFailed", {
            {"orderId", event.orderId}
        });
        // Trigger compensation: release inventory
        messageQueue.publish("CompensateInventory", {
            {"productId", event.productId}
        });
    }
}

// Product Service (listens to CompensateInventory)
void onCompensateInventory(Event event) {
    releaseInventory(event.productId);  // Rollback!
```

topic_6_microservices_architecture

Edge Cases

📌 1. Distributed Transactions (SAGA Pattern) (3/3)

Alternative: SAGA Orchestrator

Central orchestrator coordinates:

```
class OrderSagaOrchestrator {
    void executeSaga(OrderRequest request) {
        try {
            // Step 1
            productService.reserveInventory(request.productId);

            // Step 2
            paymentService.chargeCard(request.userId, request.amount);

            // Step 3
            orderService.createOrder(request);

            std::cout << "Order completed successfully!\n";

        } catch (std::exception& e) {
            // Compensate (rollback)
            std::cout << "Order failed, rolling back...\n";
            productService.releaseInventory(request.productId);
            // Payment refund if needed
        }
    }
};
```

Lesson: Use **SAGA pattern** for distributed transactions. Either choreography (events) or orchestration (coordinator).

📌 2. Network Failures (Circuit Breaker Pattern) (1/3)

Problem: Service B is down. Service A keeps calling it → wastes resources, cascading failures.

Naive Code:

```
std::string getUserProfile(int userId) {
    try {
        // Order Service is down!
        auto orders = orderService.getOrders(userId);  // ❌ Times out (5 seconds)
        return renderProfile(userId, orders);
    } catch (...) {
        return "Error loading profile";
    }
}

// If 1000 requests come in, all wait 5 seconds → 5000 seconds wasted!
```

Why It Fails:
- **Cascading failure**: Service A becomes slow because Service B is down
- **Resource exhaustion**: All threads blocked waiting for timeouts

Solution: Circuit Breaker Pattern

Like electrical circuit breaker: if too many failures, "open" circuit (stop calling).

```
class CircuitBreaker {
    enum State { CLOSED, OPEN, HALF_OPEN };

    State state = CLOSED;
    int failureCount = 0;
    int failureThreshold = 5;
    std::chrono::time_point<std::chrono::steady_clock> openedAt;

public:
    template<typename Func>
    auto call(Func func) {
        if (state == OPEN) {
            // Check if should try again
            auto now = std::chrono::steady_clock::now();
            auto elapsed = std::chrono::duration_cast<std::chrono::seconds>(now - openedAt).count();

// ...continues on next slide
```

📌 2. Network Failures (Circuit Breaker Pattern) (2/3)

```
// ...continued from previous slide
            if (elapsed < 60) {  // Wait 60 seconds before retrying
                throw std::runtime_error("Circuit breaker OPEN");
            } else {
                state = HALF_OPEN;  // Try one request
            }
        }

        try {
            auto result = func();  // Execute actual call

            // Success!
            if (state == HALF_OPEN) {
                state = CLOSED;  // Circuit recovered
                failureCount = 0;
            }

            return result;

        } catch (...) {
            failureCount++;

            if (failureCount >= failureThreshold) {
                state = OPEN;
                openedAt = std::chrono::steady_clock::now();
                std::cout << "Circuit breaker OPENED\n";
            }

            throw;
        }
    }
};

// Usage
CircuitBreaker orderServiceBreaker;

std::string getUserProfile(int userId) {
    try {
        auto orders = orderServiceBreaker.call([&]() {
            return orderService.getOrders(userId);
        });
// ...continues on next slide
```

📌 2. Network Failures (Circuit Breaker Pattern) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
        return renderProfile(userId, orders);

    } catch (...) {
        // Circuit open or service down → fallback
        return renderProfile(userId, {});  // Show profile without orders
    }
}
```

Flow:

```
State: CLOSED (normal)
  ↓ (5 failures)
State: OPEN (stop calling, return error immediately)
  ↓ (wait 60 seconds)
State: HALF_OPEN (try one request)
  ↓ (success)
State: CLOSED (recovered!)
```

Lesson: Use **Circuit Breaker** to prevent cascading failures and resource exhaustion.

---

topic_6_microservices_architecture

Edge Cases

📌 3. Data Consistency (Eventual Consistency) (1/2)

Problem: User Service updates email, but Order Service still has old email (cached or separate DB).

Scenario:

```
User updates email → User Service updates user DB
                   → Order Service still has old email in cache
                   → Sends order confirmation to OLD email!
```

Why It Happens:
- Each service has its own database
- Data duplication across services
- Network delays in propagating updates

Solution 1: Event-Driven Updates

```
// User Service
void updateEmail(int userId, const std::string& newEmail) {
    userRepository.updateEmail(userId, newEmail);

    // Publish event
    messageQueue.publish("UserEmailUpdated", {
        {"userId", userId},
        {"newEmail", newEmail}
    });
}

// Order Service (subscriber)
void onUserEmailUpdated(Event event) {
    // Update local cache/copy
    userCache.updateEmail(event.userId, event.newEmail);
    std::cout << "Order Service: Updated email for user " << event.userId << "\n";
}
```

📌 3. Data Consistency (Eventual Consistency) (2/2)

Solution 2: Query Master Service

```
// Order Service ALWAYS queries User Service for latest data
std::string getUserEmail(int userId) {
    auto response = httpClient.get("http://user-service/api/users/" + std::to_string(userId));
    return parseEmail(response);  // Always fresh!
}

void sendOrderConfirmation(int userId, Order order) {
    std::string email = getUserEmail(userId);  // Query User Service
    emailService.send(email, "Order confirmed!");
}
```

Trade-offs:
- **Event-driven**: Fast (cached), but eventual consistency
- **Query master**: Slow (network call), but always consistent

Lesson: Accept **eventual consistency** or query master service for strong consistency.

---

topic_6_microservices_architecture

Edge Cases

📌 4. Service Dependency Hell (1/2)

Problem: Service A depends on B, B depends on C, C depends on D... circular dependencies!

Naive Architecture:

```
User Service → Order Service → User Service  ← Circular!
```

Why It's Bad:
- Circular dependencies cause deadlocks
- Hard to deploy (which service first?)
- Tight coupling (not really microservices!)

Solution 1: Break Circular Dependency

```
❌ BAD:
User Service ──> Order Service
     ↑                 │
     └─────────────────┘

✅ GOOD:
User Service
     ↓
Order Service  (no dependency back to User Service)
```

📌 4. Service Dependency Hell (2/2)

Solution 2: Use Events (Loose Coupling)

```
// User Service doesn't call Order Service directly
void registerUser(const User& user) {
    userRepository.save(user);

    messageQueue.publish("UserRegistered", {
        {"userId", user.id}
    });

    // Order Service listens to this event (no direct dependency!)
}

// Order Service
void onUserRegistered(Event event) {
    // Initialize empty order history for new user
    createOrderHistory(event.userId);
}
```

Solution 3: Shared Kernel (Last Resort)

```
Create shared library with common models:
  - User model
  - Order model
  - Common utilities

All services depend on shared library (read-only).
```

Lesson: **Avoid circular dependencies.** Use events for loose coupling.

📌 5. Debugging Distributed Systems (Distributed Tracing) (1/3)

Problem: Request goes through 10 services. Where did it fail?

Scenario:

```
API Gateway → User Service → Order Service → Payment Service → Shipping Service
                                                                  ↑
                                                            (failed here!)
```

How to know Payment Service failed? Logs are scattered across services!

📌 5. Debugging Distributed Systems (Distributed Tracing) (2/3)

Solution: Distributed Tracing (Correlation IDs)

```
class Request {
public:
    std::string traceId;  // Unique ID for this request
    std::string spanId;   // ID for this service's work
    std::string parentSpanId;  // Previous service's spanId
};

// API Gateway (entry point)
void handleRequest(HttpRequest req) {
    Request request;
    request.traceId = generateUUID();  // e.g., "abc-123-def"
    request.spanId = generateUUID();
    request.parentSpanId = "";

    log("[TraceID: " + request.traceId + "] API Gateway received request");

    forwardToUserService(request);
}

// User Service
void handleRequest(Request req) {
    std::string mySpanId = generateUUID();

    log("[TraceID: " + req.traceId + "][SpanID: " + mySpanId + "] User Service processing");

    Request newReq;
    newReq.traceId = req.traceId;  // Keep same traceId!
    newReq.spanId = generateUUID();
    newReq.parentSpanId = mySpanId;

    forwardToOrderService(newReq);
}

// Payment Service (where it fails)
void handleRequest(Request req) {
    log("[TraceID: " + req.traceId + "] Payment Service processing");

    try {
        chargeCard();
// ...continues on next slide
```

📌 5. Debugging Distributed Systems (Distributed Tracing) (3/3)

[Code continues across slides]

```
// ...continued from previous slide
    } catch (...) {
        log("[TraceID: " + req.traceId + "] ERROR: Payment failed!");  // ← Easy to find!
        throw;
    }
}
```

Now search logs for traceId "abc-123-def":

```
[TraceID: abc-123-def] API Gateway received request
[TraceID: abc-123-def] User Service processing
[TraceID: abc-123-def] Order Service processing
[TraceID: abc-123-def] Payment Service processing
[TraceID: abc-123-def] ERROR: Payment failed!  ← Found it!
```

Tools: Zipkin, Jaeger, OpenTelemetry

Lesson: Use **distributed tracing** with correlation IDs to debug across services.

📌 6. Database per Service (Data Duplication) (1/2)

Problem: Multiple services need user data. Do we duplicate it?

Scenario:

```
User Service has:      {id: 1, name: "Alice", email: "alice@example.com"}
Order Service needs:   User's name for order confirmation
Shipping Service needs: User's email for tracking updates
```

Option 1: Query User Service (Strong Consistency)

```
// Order Service
void createOrder(int userId, int productId) {
    // Query User Service for user data
    auto user = httpClient.get("http://user-service/api/users/" + std::to_string(userId));

    Order order = {userId, productId, user.name};  // Use fresh data
    orderRepository.save(order);
}
```

Pros: Always consistent

Cons: Slow (network call), User Service becomes bottleneck

Option 2: Cache User Data Locally (Eventual Consistency)

```
// Order Service maintains local cache
class OrderService {
    std::map<int, User> userCache;  // Local cache

    void onUserUpdated(Event event) {
        // Update cache when User Service publishes event
        userCache[event.userId] = event.user;
    }

    void createOrder(int userId, int productId) {
        User user = userCache[userId];  // Fast! No network call
        Order order = {userId, productId, user.name};
        orderRepository.save(order);
    }
};
```

📌 6. Database per Service (Data Duplication) (2/2)

Pros: Fast (no network call)
Cons: Eventual consistency (cache might be stale)

Lesson: **Trade-off between consistency and performance.** Use caching + events for eventual consistency.

---

topic_6_microservices_architecture

Code Examples

📌 1. Simple REST Microservice (1/6)

Goal: Create a basic User Service with REST API.

Output:

```
#include <iostream>
#include <string>
#include <map>
#include <sstream>
// Note: In real app, use library like cpp-httplib, Crow, or Boost.Beast

// ═══════════════════════════════════════════════════════════════
//  USER MODEL
// ═══════════════════════════════════════════════════════════════

struct User {
    int id;
    std::string name;
    std::string email;
};

// ═══════════════════════════════════════════════════════════════
//  USER REPOSITORY (Database Access)
// ═══════════════════════════════════════════════════════════════

class UserRepository {
    std::map<int, User> users;
    int nextId = 1;

public:
    UserRepository() {
        // Seed data
        users[1] = {1, "Alice", "alice@example.com"};
        users[2] = {2, "Bob", "bob@example.com"};
        nextId = 3;
    }

    User getById(int id) {
        if (users.find(id) == users.end()) {
            throw std::runtime_error("User not found");
        }
// ...continues on next slide
```

📌 1. Simple REST Microservice (2/6)

```
// ...continued from previous slide
        return users[id];
    }

    std::vector<User> getAll() {
        std::vector<User> result;
        for (auto& [id, user] : users) {
            result.push_back(user);
        }
        return result;
    }

    User create(const std::string& name, const std::string& email) {
        User user = {nextId++, name, email};
        users[user.id] = user;
        return user;
    }

    void update(int id, const std::string& name, const std::string& email) {
        if (users.find(id) == users.end()) {
            throw std::runtime_error("User not found");
        }
        users[id].name = name;
        users[id].email = email;
    }

    void remove(int id) {
        users.erase(id);
    }
};

// ═══════════════════════════════════════════════════════════
//  USER SERVICE (Business Logic)
// ═══════════════════════════════════════════════════════════

class UserService {
    UserRepository repository;

public:
    User getUserById(int id) {
        return repository.getById(id);
// ...continues on next slide
```

topic_6_microservices_architecture

Code Examples

📌 1. Simple REST Microservice (3/6)

```
// ...continued from previous slide
    }

    std::vector<User> getAllUsers() {
        return repository.getAll();
    }

    User createUser(const std::string& name, const std::string& email) {
        // Validation
        if (name.empty()) {
            throw std::invalid_argument("Name is required");
        }
        if (email.find('@') == std::string::npos) {
            throw std::invalid_argument("Invalid email format");
        }

        return repository.create(name, email);
    }

    void updateUser(int id, const std::string& name, const std::string& email) {
        repository.update(id, name, email);
    }

    void deleteUser(int id) {
        repository.remove(id);
    }
};

// ═══════════════════════════════════════════════════════════════
//  REST API CONTROLLER (HTTP Interface)
// ═══════════════════════════════════════════════════════════════

class UserController {
    UserService service;

public:
    // Simulate HTTP request handling
    std::string handleRequest(const std::string& method, const std::string& path) {
        try {
            if (method == "GET" && path == "/api/users") {
                return handleGetAllUsers();
// ...continues on next slide
```

topic_6_microservices_architecture

Code Examples

📌 1. Simple REST Microservice (4/6)

```
// ...continued from previous slide
            } else if (method == "GET" && path.find("/api/users/") == 0) {
                int userId = std::stoi(path.substr(11));  // Extract ID
                return handleGetUser(userId);
            } else if (method == "POST" && path == "/api/users") {
                return handleCreateUser("Charlie", "charlie@example.com");  // Simulated
            } else {
                return "{\"error\": \"Not found\"}";
            }
        } catch (std::exception& e) {
            return "{\"error\": \"" + std::string(e.what()) + "\"}";
        }
    }

private:
    std::string handleGetAllUsers() {
        auto users = service.getAllUsers();
        std::ostringstream json;
        json << "[";
        for (size_t i = 0; i < users.size(); i++) {
            json << "{\"id\":" << users[i].id
                 << ",\"name\":\"" << users[i].name << "\""
                 << ",\"email\":\"" << users[i].email << "\"}";
            if (i < users.size() - 1) json << ",";
        }
        json << "]";
        return json.str();
    }

    std::string handleGetUser(int userId) {
        User user = service.getUserById(userId);
        std::ostringstream json;
        json << "{\"id\":" << user.id
             << ",\"name\":\"" << user.name << "\""
             << ",\"email\":\"" << user.email << "\"}";
        return json.str();
    }

    std::string handleCreateUser(const std::string& name, const std::string& email) {
        User user = service.createUser(name, email);
        std::ostringstream json;
// ...continues on next slide
```

topic_6_microservices_architecture

Code Examples

📌 1. Simple REST Microservice (5/6)

[Code continues across slides]

```
// ...continued from previous slide
        json << "{\"id\":" << user.id
             << ",\"name\":\"" << user.name << "\""
             << ",\"email\":\"" << user.email << "\"}";
        return json.str();
    }
};

// ═══════════════════════════════════════════════════════════
//  MAIN (Microservice Entry Point)
// ═══════════════════════════════════════════════════════════

int main() {
    std::cout << "═══════════════════════════════════════════════════\n";
    std::cout << "  User Microservice (REST API)                    \n";
    std::cout << "  Running on port 8001                            \n";
    std::cout << "═══════════════════════════════════════════════════\n\n";

    UserController controller;

    // Simulate HTTP requests
    std::cout << "=== GET /api/users ===\n";
    std::cout << controller.handleRequest("GET", "/api/users") << "\n\n";

    std::cout << "=== GET /api/users/1 ===\n";
    std::cout << controller.handleRequest("GET", "/api/users/1") << "\n\n";

    std::cout << "=== POST /api/users (Create Charlie) ===\n";
    std::cout << controller.handleRequest("POST", "/api/users") << "\n\n";

    std::cout << "=== GET /api/users (After Create) ===\n";
    std::cout << controller.handleRequest("GET", "/api/users") << "\n\n";

    std::cout << "═══════════════════════════════════════════════════\n";
    std::cout << "  Key Points:                                      \n";
    std::cout << "  1. Independent service with own data            \n";
    std::cout << "  2. REST API for communication                   \n";
    std::cout << "  3. Can deploy/scale independently               \n";
    std::cout << "═══════════════════════════════════════════════════\n";

    return 0;
```

topic_6_microservices_architecture

Code Examples

📌 1. Simple REST Microservice (6/6)

```
['═══════════════════════════════════════════════════════════\n  User Microservice (REST API)\n  Running on port 8001\n═══════
```

What You Learned

Key Takeaways

- ✓ What is Microservices Architecture?
- ✓ Simple Explanation: City vs Village Analogy
- ✓ Monolithic vs Microservices
- ✓ Monolithic Application

→ Swipe to learn →